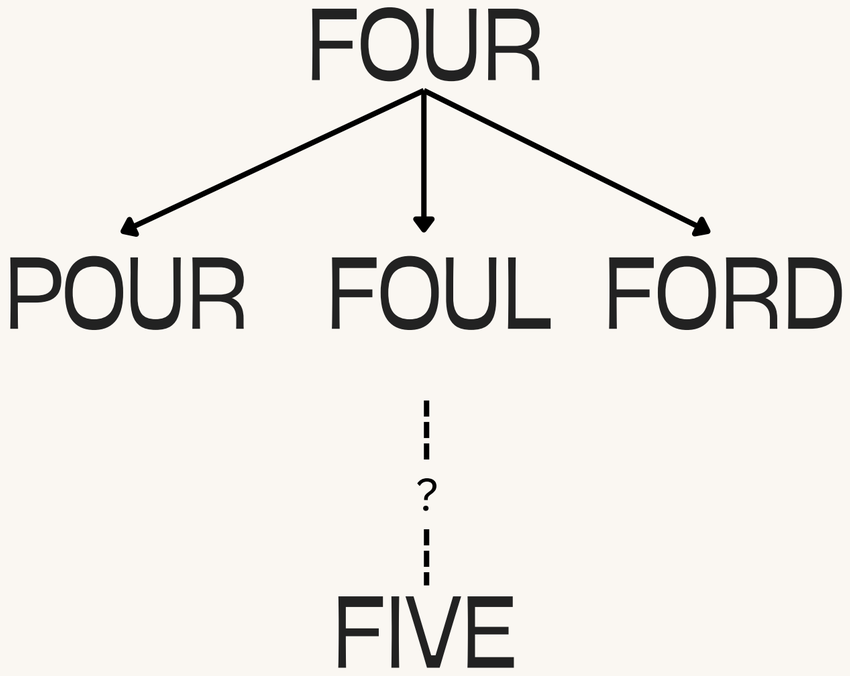
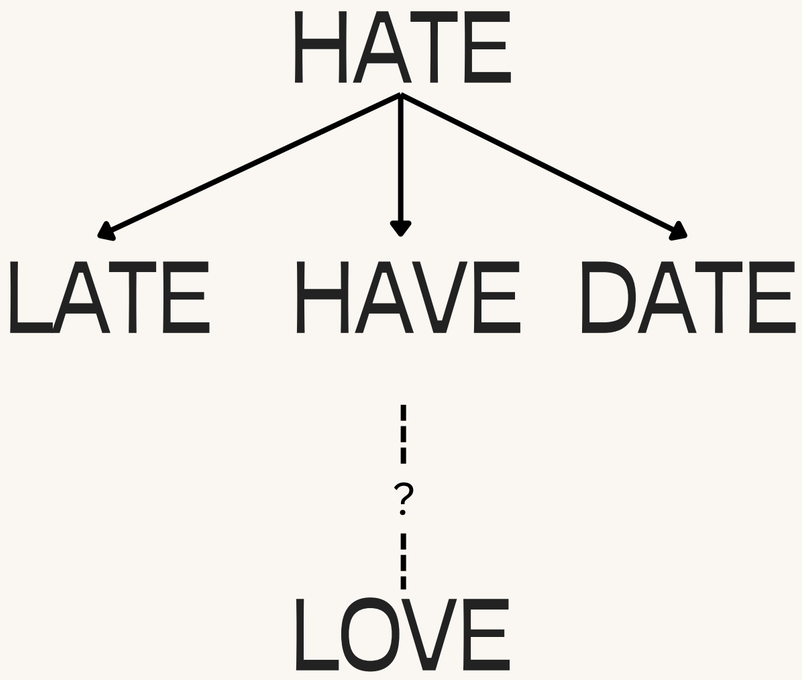
WELCOME!

You will need groups of 4 for today's activity.

First, look at these Word Ladders. The goal is to transform one word into another by changing one letter at a time, creating a valid word at each step. Which branch would you choose to investigate first? Can you explain why?

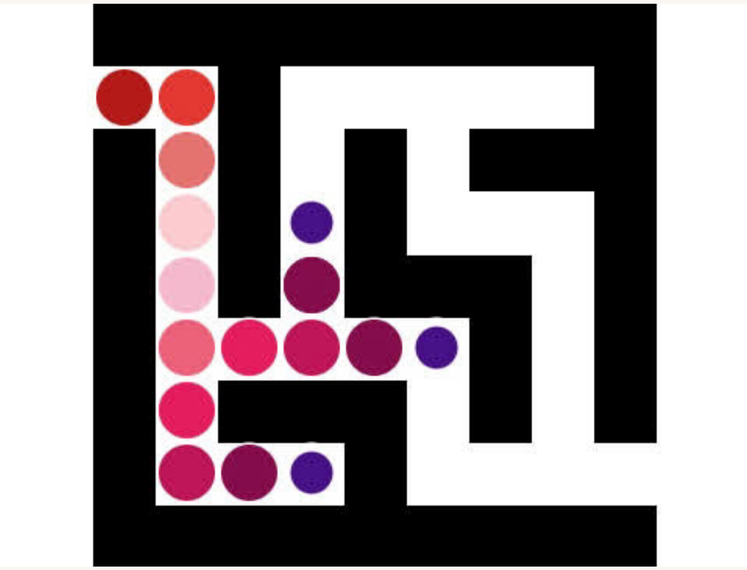




SEARCH PART 2

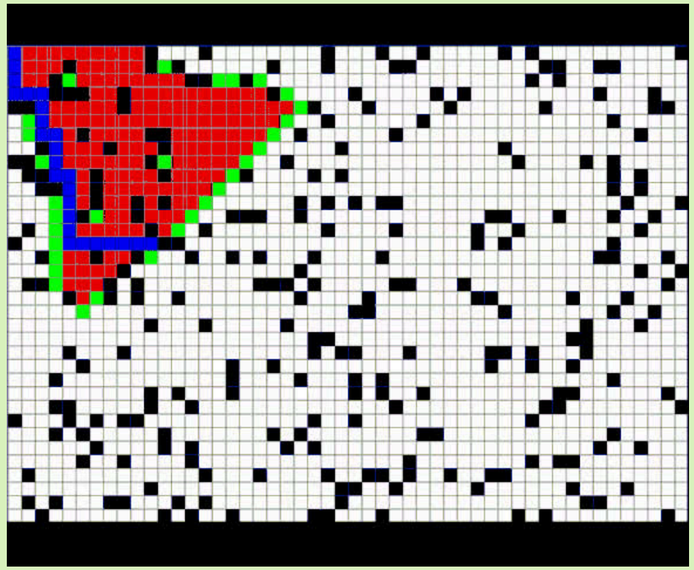
UNINFORMED

- The agent has no additional information other than the goal
- Systematically explores every option



INFORMED

- The agent can estimate how far it is from the goal
- Uses additional information to guide the search process
- Cost-based and goal-directed
- Prioritizing more promising paths, avoids unlikely paths



HEURISTICS

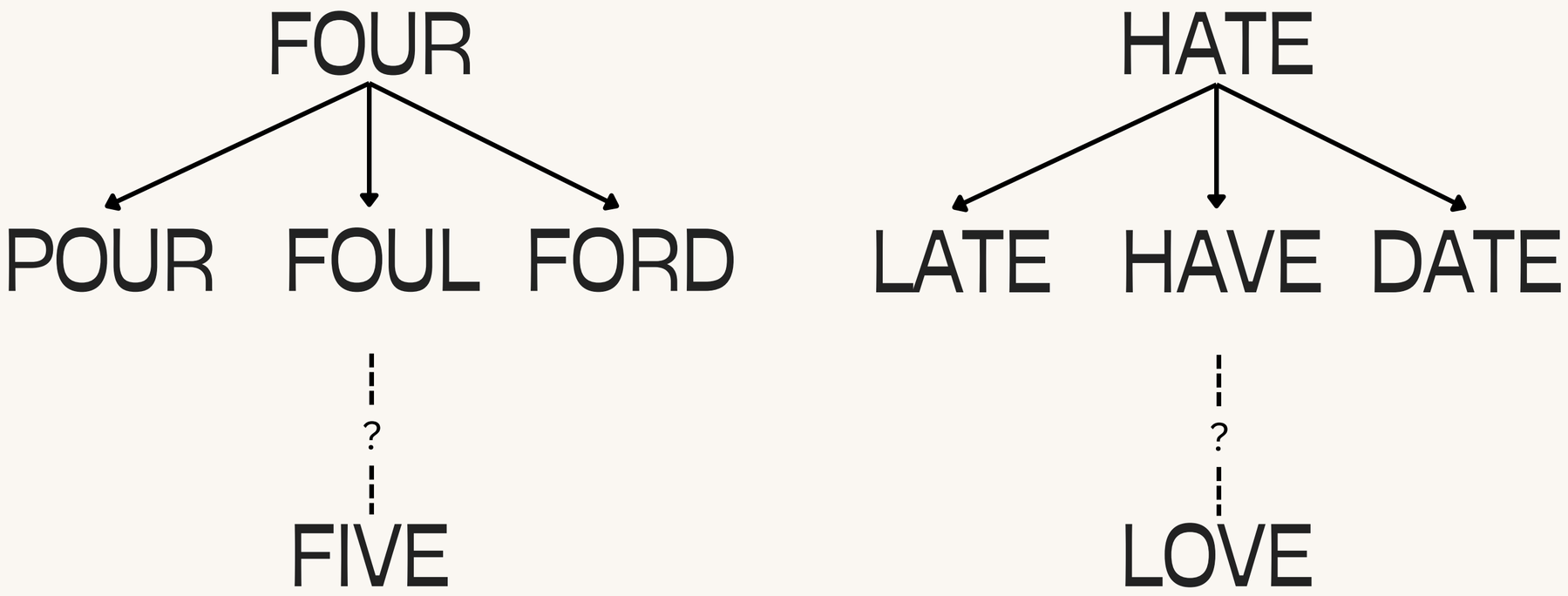
h(n) = the estimated cost of the cheapest path from the state at node n to a goal state

Uses only local information to compare how far a state is from a goal state

HEURISTICS

h(n) = the estimated cost of the cheapest path from the state at node n to a goal state

Uses only local information to compare how far a state is from a goal state



LET'S TRY IT!

GET_SUCCESSORS

You have a dictionary of successors:

A: B, C means B and C are successors of A

Input: a state ID
Output: all successors of that state in the order listed

IS_GOAL + HEURISTIC

You know the goal state(s)/conditions. You also estimate the distance a state is from the goal

Input: a state ID
Output: Yes or No if that state is a goal OR
Output: a numerical estimate of how far a state is from the goal

OPEN LIST

You keep track of what states we have yet to visit, as well as the parents so we can backtrack the path.

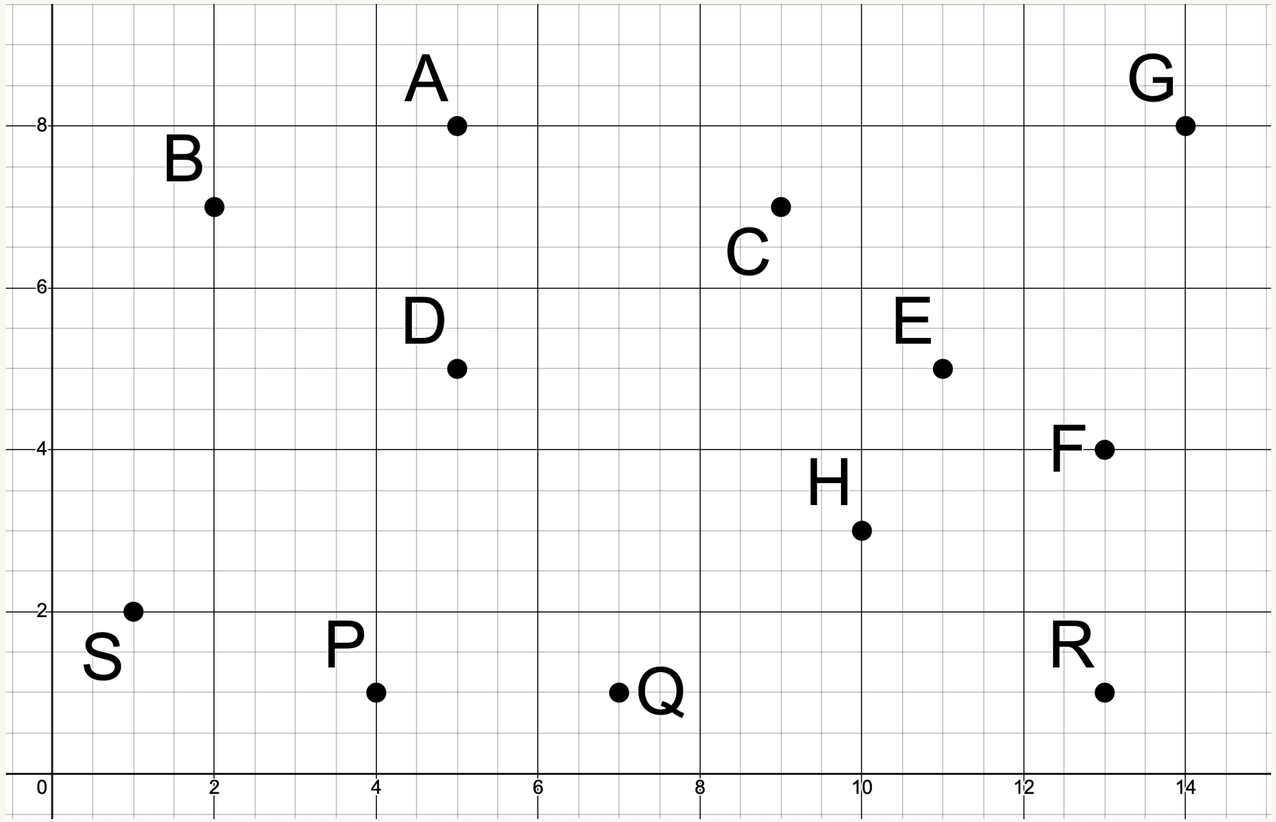
Input: a state ID
Output: the next state to visit, and where that state came from

THE ALGORITHM

You use all the helper functions to create a search tree and find the goal state

Input: the starting state
Output: a list of actions for how to get from the start state to the goal state

WATCH ME




```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

Start Node: S

Goal Node: G

A: none
B: (A, 5)
C: (A, 6), (F, 8)
D: (B, 6), (C, 7), (E, 7)
E: (H, 4), (R, 7)
F: (G, 6)
G: none
H: (P, 9), (Q, 6)
P: (Q, 4)
Q: none
R: (F, 4)
S: (D, 8), (E, 14), (P, 4)

WATCH ME

```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

LET'S TRY IT!

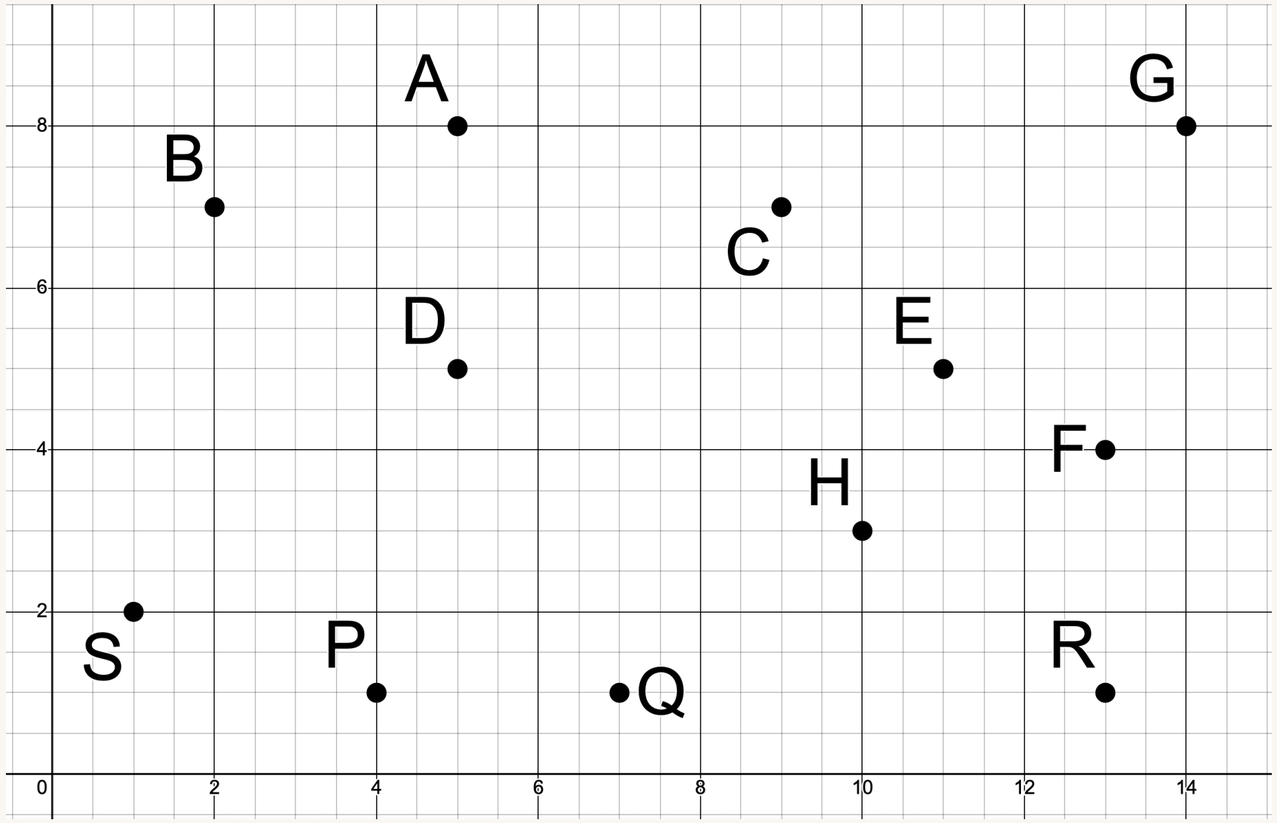
PATH:
S → D → C → F → G

VISITED:
S, D, E, P, C, Q, B, H, F, A, R, G

COST:
29

Start Node: S

Goal Node: G



WORLD1

LET'S TRY IT!

Start Node: A

LET'S TRY IT!

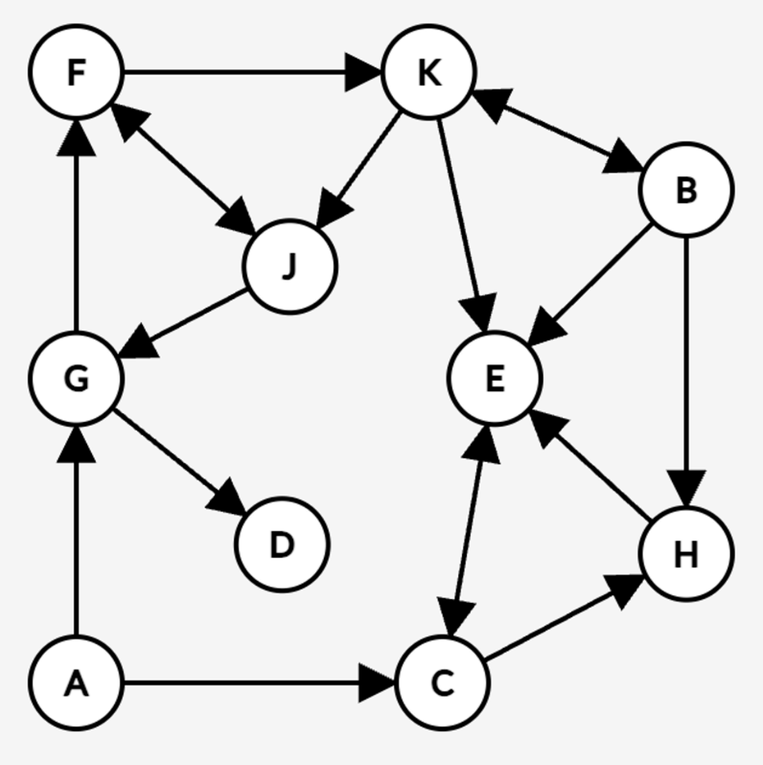
PATH:
A → G → F → J

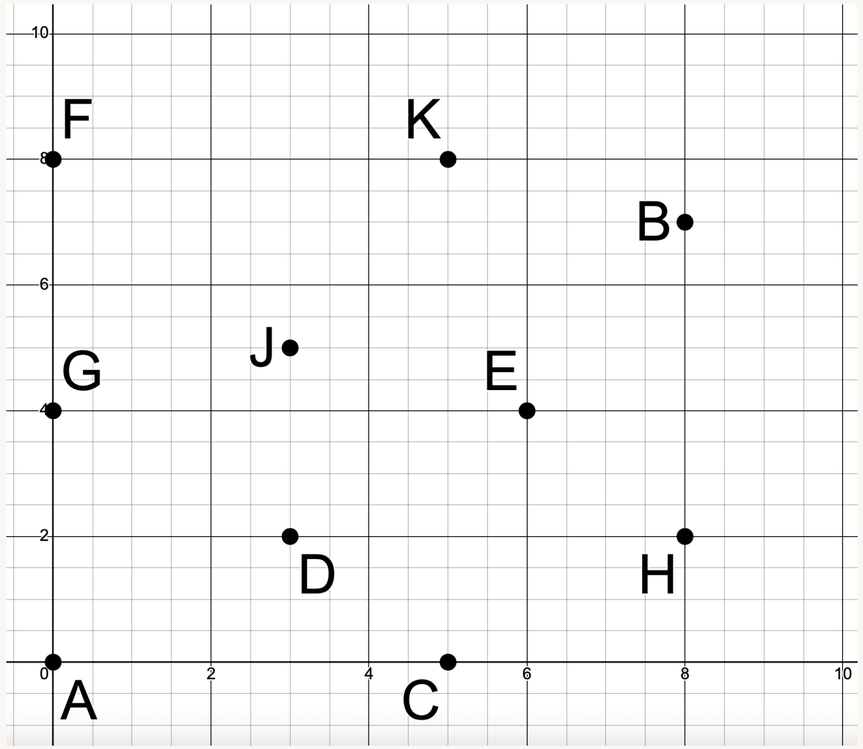
VISITED:
A, G, D, F

COST:
17

Start Node: A

Goal Node: J





WORLD2

LET'S TRY IT!

Start Node: 3

Remember: a solution is a sequence of actions which transforms the start state into a goal state

WORLD3

LET'S TRY IT!

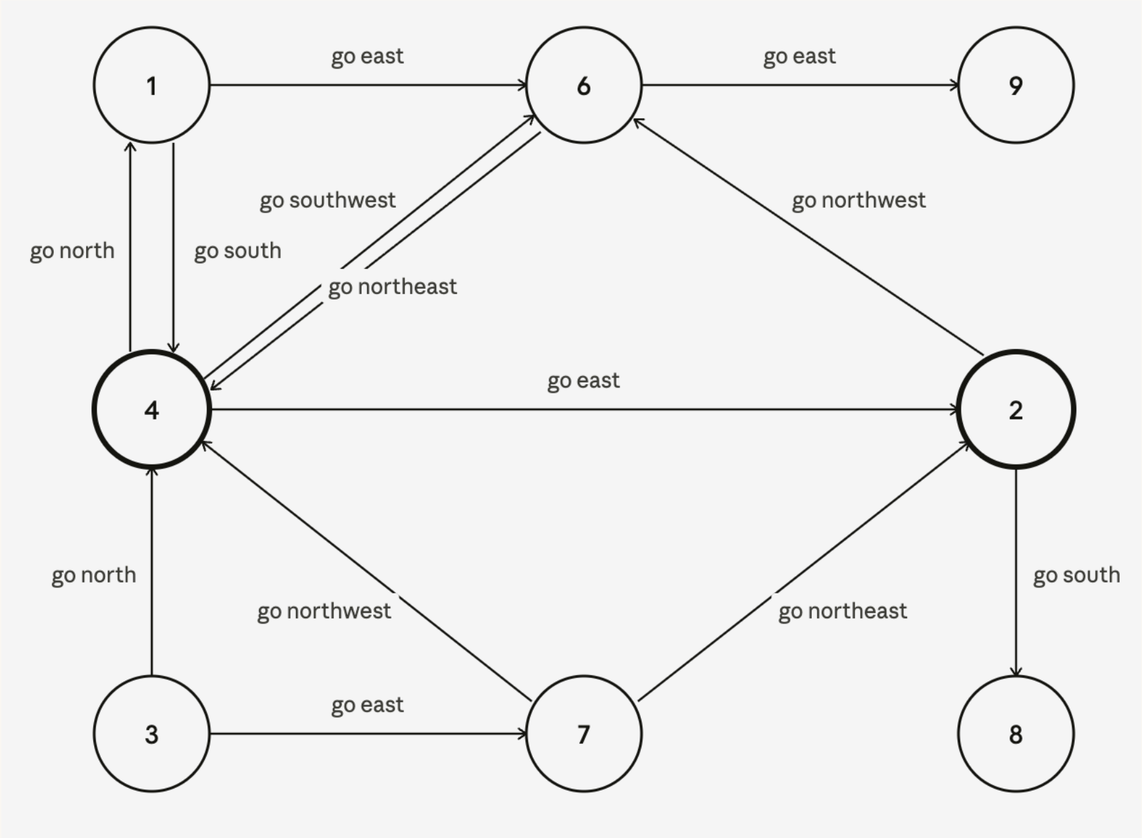
PATH:
go north → go northeast → go east

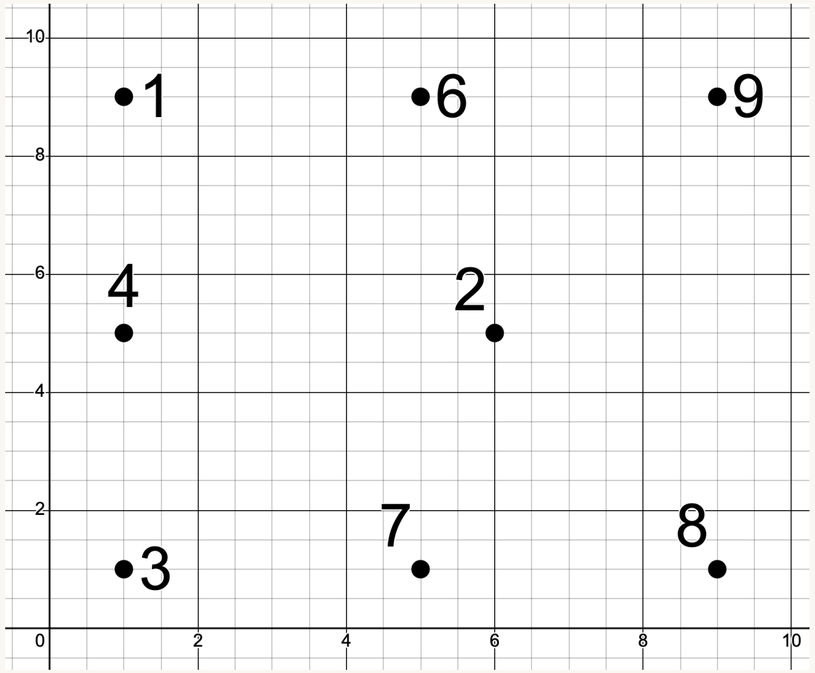
VISITED:
3, 4, 6

COST:
19

Start Node: 3

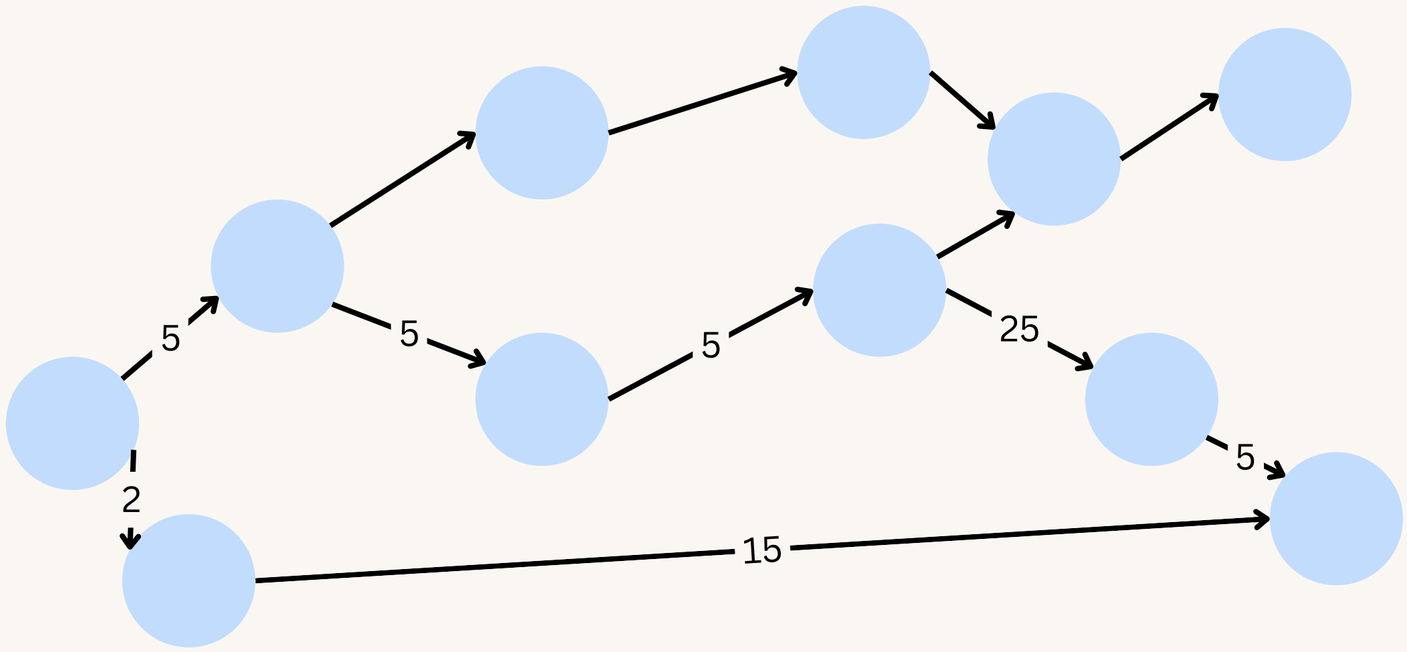
Goal Node: 9



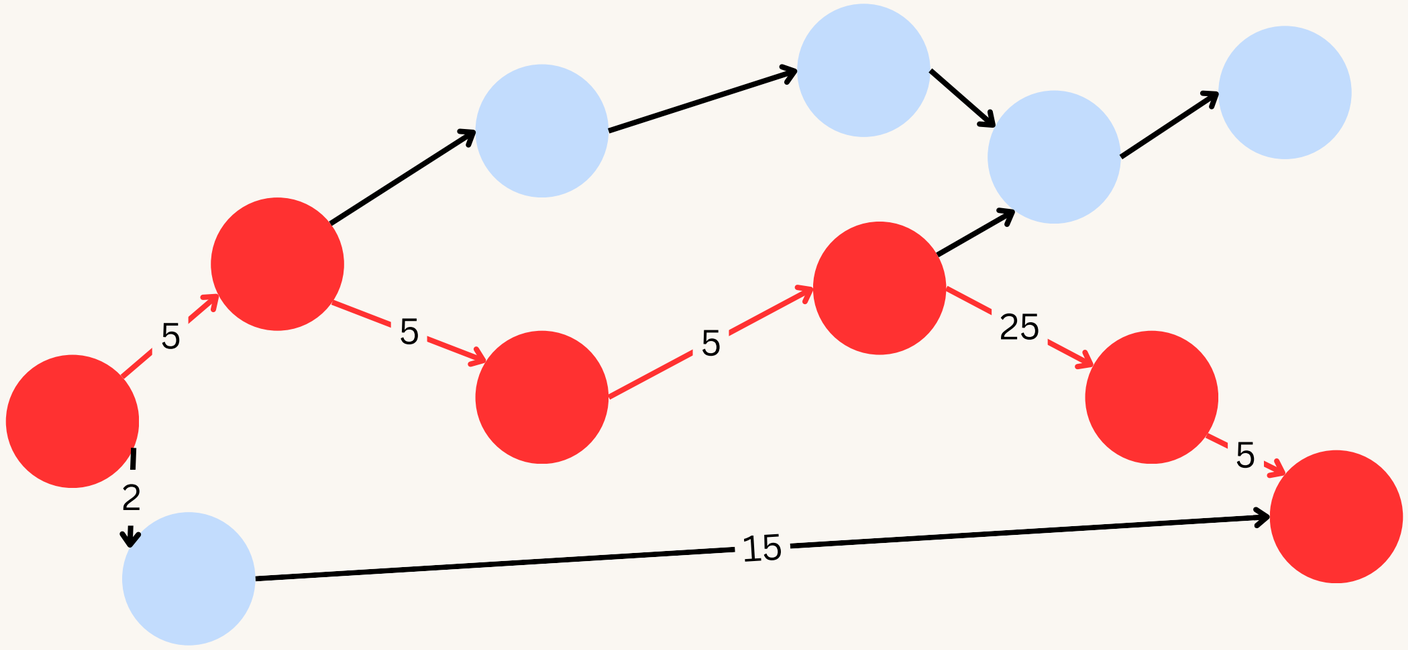


WORLD3

PROBLEMS?



PROBLEMS?



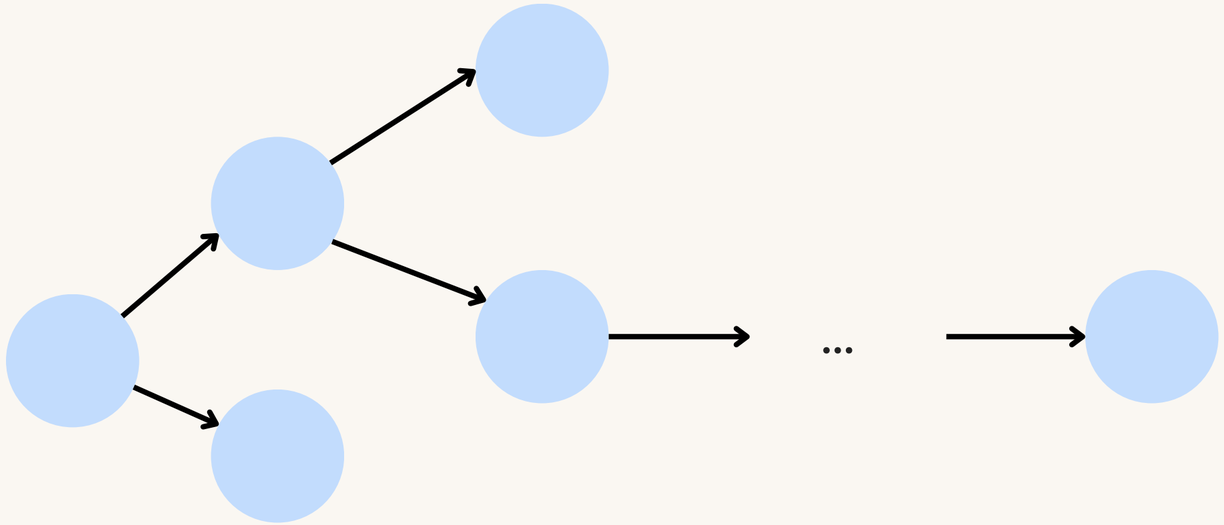
THE BEST OF BOTH WORLDS

Pick states that have the lowest estimated total cost

h(n) = estimated cost from n to the goal
g(n) = known cost from start to n

Evaluation function:
f(n) = g(n) + h(n)

This is A* search



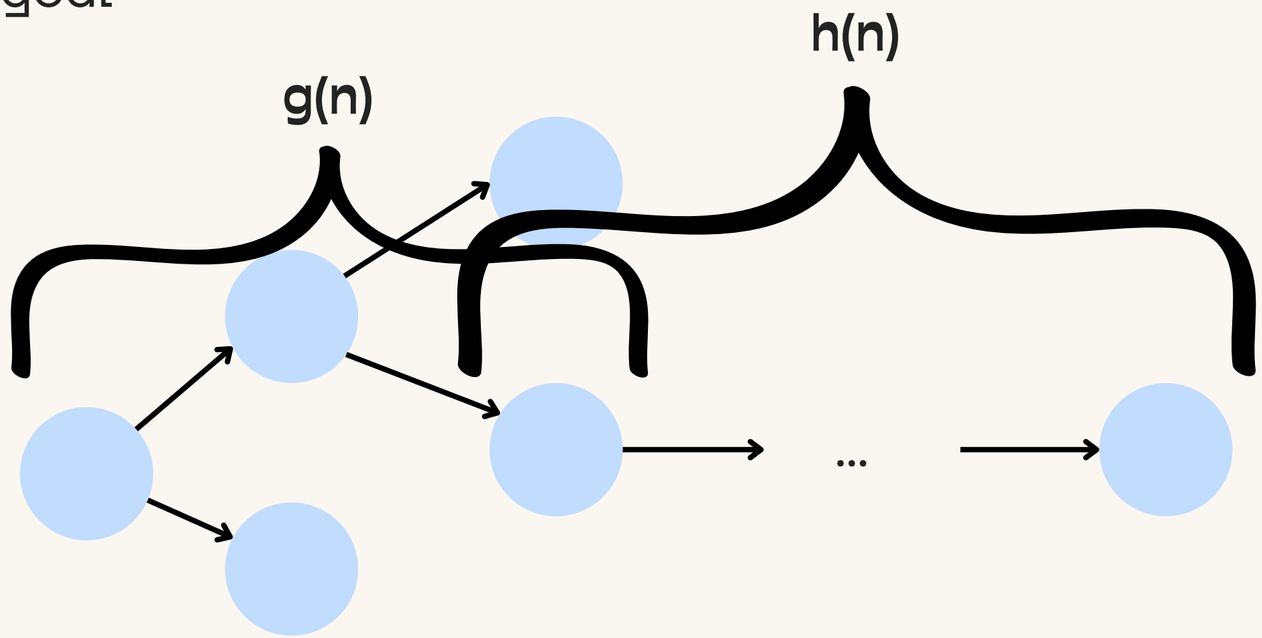
THE BEST OF BOTH WORLDS

Pick states that have the lowest estimated total cost

h(n) = estimated cost from n to the goal
g(n) = known cost from start to n

Evaluation function:
f(n) = g(n) + h(n)

This is A* search



LET'S TRY IT!

GET_SUCCESSORS

You have a dictionary of successors:

A: B, C means B and C are successors of A

Input: a state ID
Output: all successors of that state in the order listed

IS_GOAL + HEURISTIC

You know the goal state(s)/conditions. You also estimate the distance a state is from the goal

Input: a state ID
Output: Yes or No if that state is a goal
Output: a numerical estimate of how far a state is from the goal

OPEN LIST

You keep track of what states we have yet to visit, as well as the parents so we can backtrack the path.

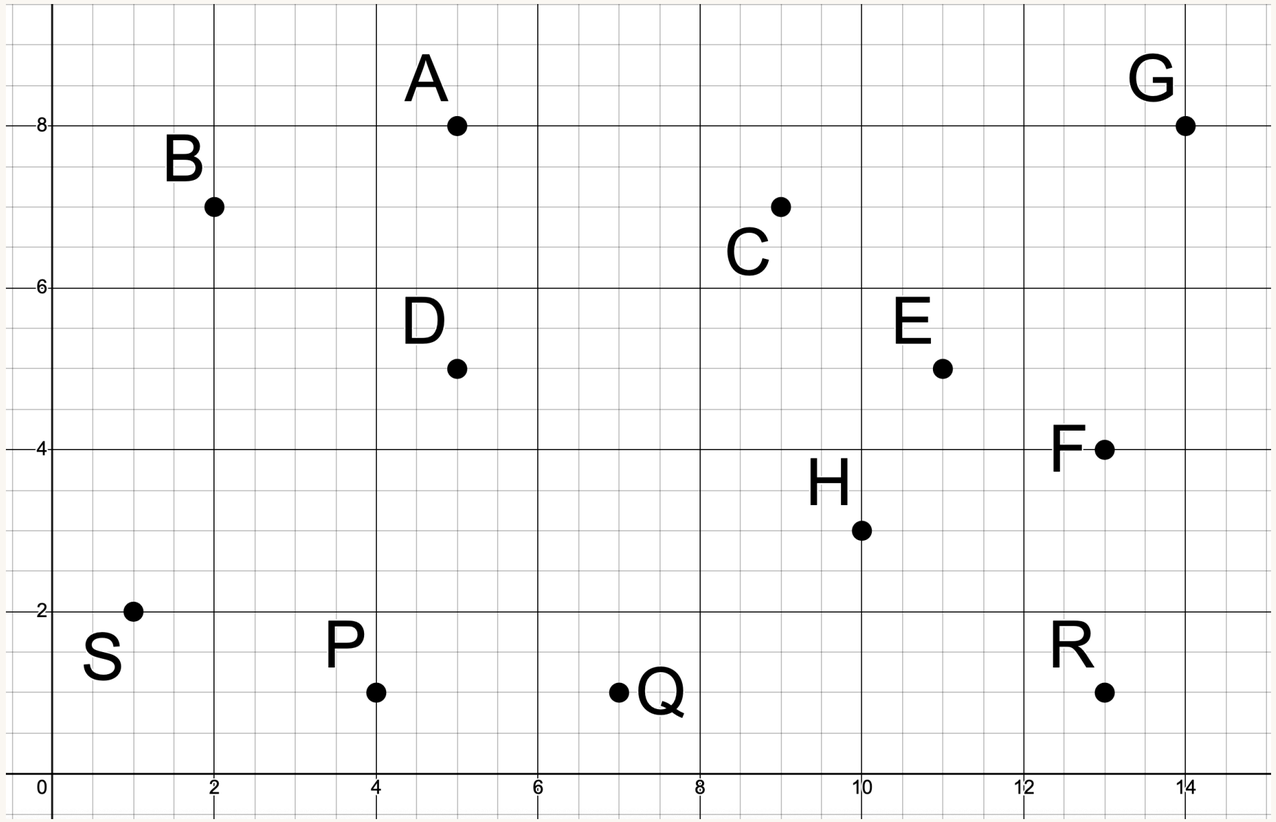
Input: a state ID
Output: the next state to visit, and where that state came from

THE ALGORITHM

You use all the helper functions to create a search tree and find the goal state

Input: the starting state
Output: a list of actions for how to get from the start state to the goal state

WATCH ME




```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

Start Node: S

Goal Node: G

A: none
B: (A, 5)
C: (A, 6), (F, 8)
D: (B, 6), (C, 7), (E, 7)
E: (H, 4), (R, 7)
F: (G, 6)
G: none
H: (P, 9), (Q, 6)
P: (Q, 4)
Q: none
R: (F, 4)
S: (D, 8), (E, 14), (P, 4)

WATCH ME

```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

LET'S TRY IT!

Start Node: A

LET'S TRY IT!

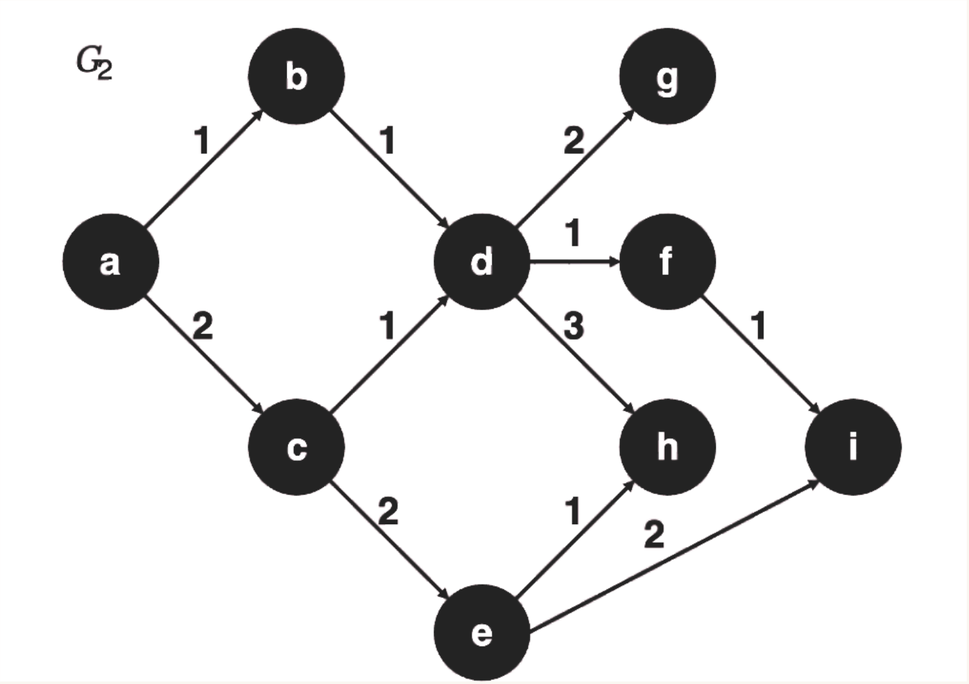
PATH:
A → C → E → I

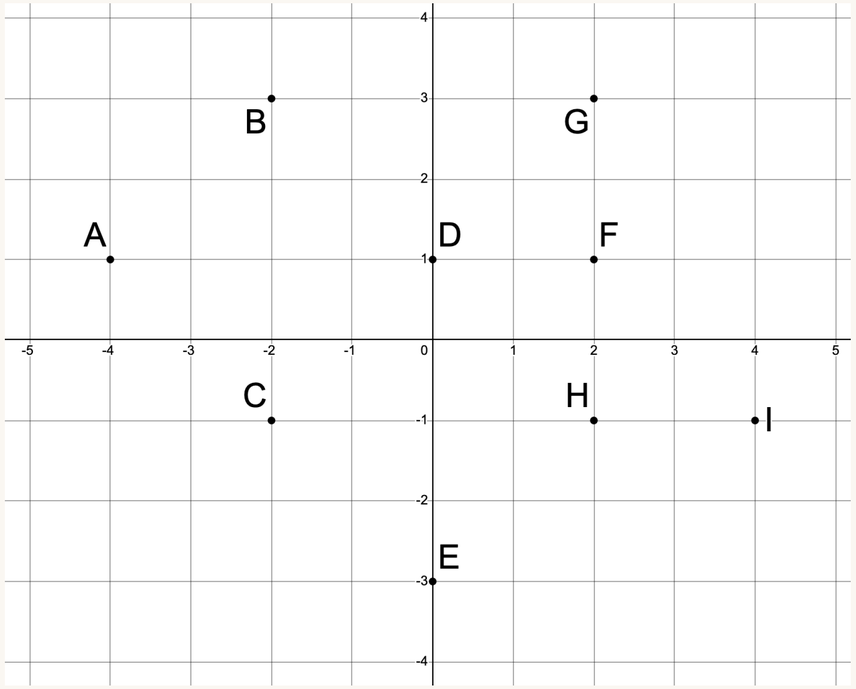
VISITED:
A, C, E, B, D, I

COST:
18

Start Node: A

Goal Node: I





WORLD4

LET'S TRY IT!

Start Node: C

LET'S TRY IT!

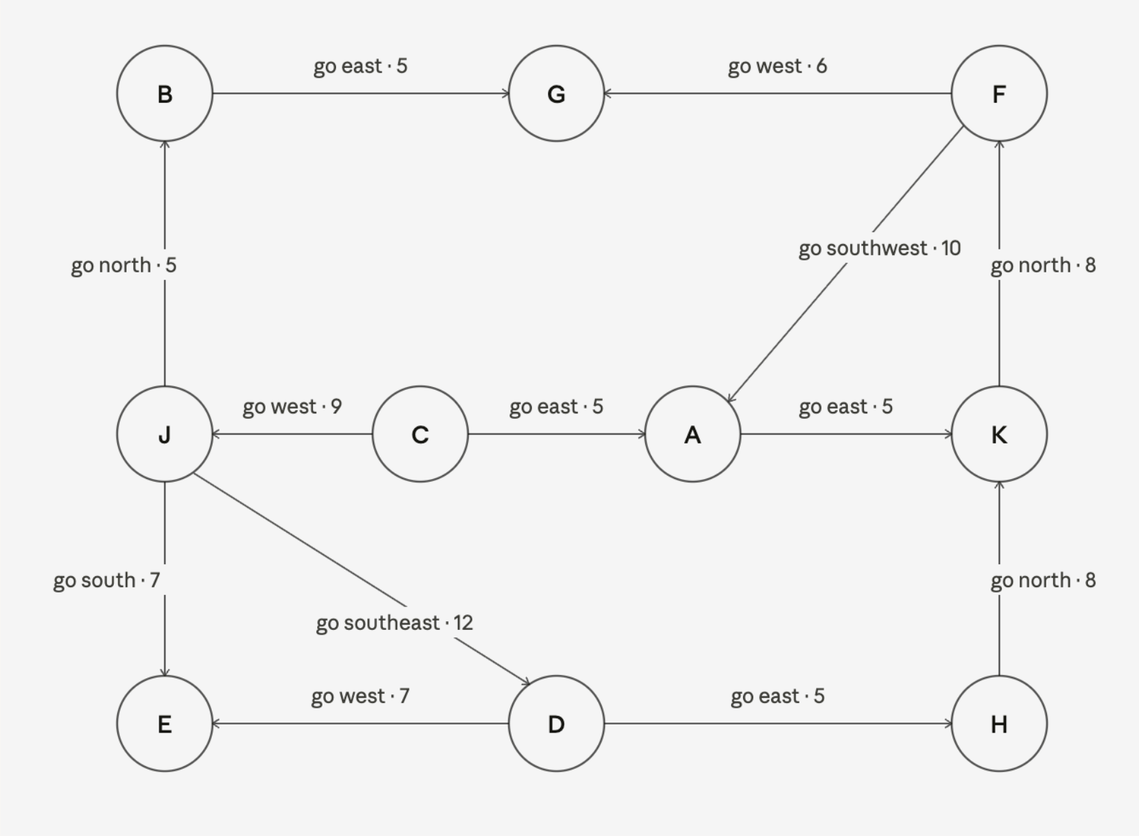
PATH:
C → J→ B → G

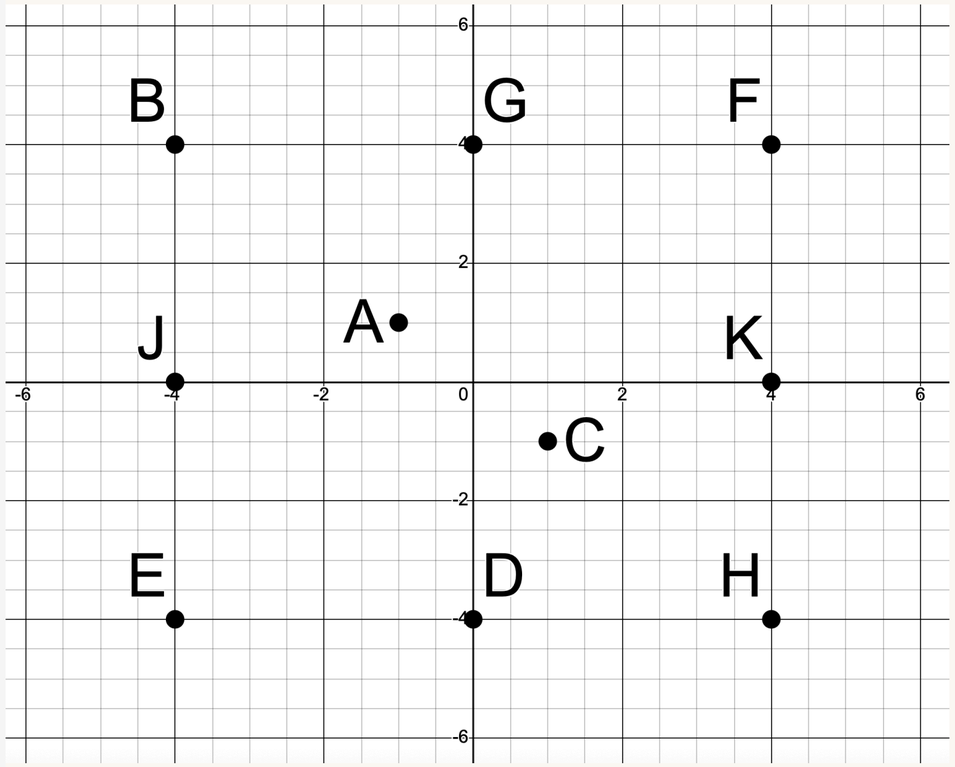
VISITED:
C, A, J, K, B, G

COST:
19

Start Node: C

Goal Node: G





WORLD5

THOUGHTS?

1. How did your group organize information across the roles?
2. Was there a role that was easier/harder than expected?
3. In what ways was your role limited?
4. How did changing the worlds change the group's behavior?
5. How did changing the Frontier's data structure and adding heuristics change the groups behavior?
6. If you had to explain this activity to a student who missed class, how would you summarize its main goal and what you learned from it?

RECAP

01 GREEDY SEARCH

Strategy: expand the node with the lowest estimated cost to the goal

02 A* SEARCH

Strategy: expand the node with the lowest estimated total path length to the goal

03 HEURISTICS

Admissibility: never over-estimates the true cost
$h(n) \leq c(n)$

Consistency: monotonic, obeys the triangle inequality
$h(A) - h(B) \leq c(A, B)$